Parallel Differentiable Mesh Rasterization for Multi-View Shape Optimization

Alejandro Amat

URL: `https://alejandroamat.github.io/15418-Differentiable-Mesh-Rasterizer/`

Summary

The idea is to implement an optimized parallel differentiable mesh and texture rasterizer for multi-view image input. First, creating a sequential baseline in C++, and then an optimized CUDA C++ GPU rasterizer to further analyze the forward and backward passes, subdivision, Laplacian smoothing, fused kernels, multi-GPU & single-GPU batching, and a CUDA Graphs implementation in terms of execution time, throughput, and memory bandwidth. Single-GPU performance will be measured on the GHC machines, and for multi-GPU experiments I will use my lab's server with 8× NVIDIA RTX A4000 GPUs.

Background

A differentiable mesh & texture rasterizer is a differentiable system that simulates traditional rasterization with the objective of obtaining the mesh (3D object) and its associated texture (advanced pipelines include more features, such as position, lighting, PBR materials, etc.) from a series of multi-view images. That is, given renders of an element from different views, it should be able to recover a mesh and a texture. Traditional pipelines do not allow this, as they use hard triangle coverage and z-buffers, which are hard thresholds that are only 1 for visible triangles and 0 for non-visible ones; this does not allow for continuous functions to differentiate and accumulate gradients through. The differentiable rasterizer simulates this through soft rasterization, which essentially replaces the hard triangle coverage with an accumulation of the faces for each pixel based on their distance. The z-buffer is replaced by the accumulation of the texture color of the faces based on their coverage and depth. By doing this, the gradients are smooth and enable good reconstruction.

The optimization starts by using an N-face spherical mesh as the optimizable mesh, along with its UV texture. Then I take a batch of the multi-view ground-truth (GT) renders, perform a forward pass of the optimizable mesh, and compare the render targets with the current renders of the optimizable mesh. Backward propagation is performed, regularizers are applied (Laplacian smoothing, in order to avoid face degeneration and collapse), and then an optimizer step is performed. In order to have smooth convergence, I start with a low-poly sphere that is subdivided after certain iterations to obtain first a coarse shape and, in later iterations, more detail. The same happens with the learning rate and the Laplacian regularizer weight, which follow a schedule to provide smooth optimization at the beginning and enable further details at the end.

There is room for parallelism in the following stages:
Forward: Similar to any rasterization pipeline, there is high potential parallelism both along the

primitive axis and the pixel axis. Not only that, there is also potential parallelism in the batch. The optimizer steps after a complete batch has finished, so the forward pass can process multiple views of the batch in the same step to enhance GPU occupancy (for instance, by using the batch as a third axis in kernel launches), or even use multi-GPU parallelism to be able to increase work-load per gpu for instance, being able to work at larger resolutions at a fixed time.
Backward: Similarly, the same applies to the backward pass, with multiple approaches to the accumulation of gradients, having the atomic accumulation of gradients, or diverse strategies such as warp reductions can be applied for less contention. Or having different buffers per view and finally doing a reduction. There are several methods that can be tried and will definitely depend ion the size of the mesh. That is why I will do profiling both with heavy mesh and a very simlple one. Batching will probably also increase the contention in these passes, which allows for an exhaustive study.
Subdivision, Laplacian smoothing, optimizer, and loss computation: there is a big axis for parallelization too, as they are done per vertex or per pixel (loss) and can make use of SIMD capabilities. Subdivision is more interesting, because edges are shared between faces, so other structures and prefix-sum strategies can be used to make it parallel too. For texture optimization, color is computed similarly to alpha in the mesh-only pipeline, but the backward pass allows for an interesting analysis, as many pixels get information from the same texels, thus creating contention there. Similarly to the atomics for the vertices, different strategies can be implemented to reduce texel gradient contention.

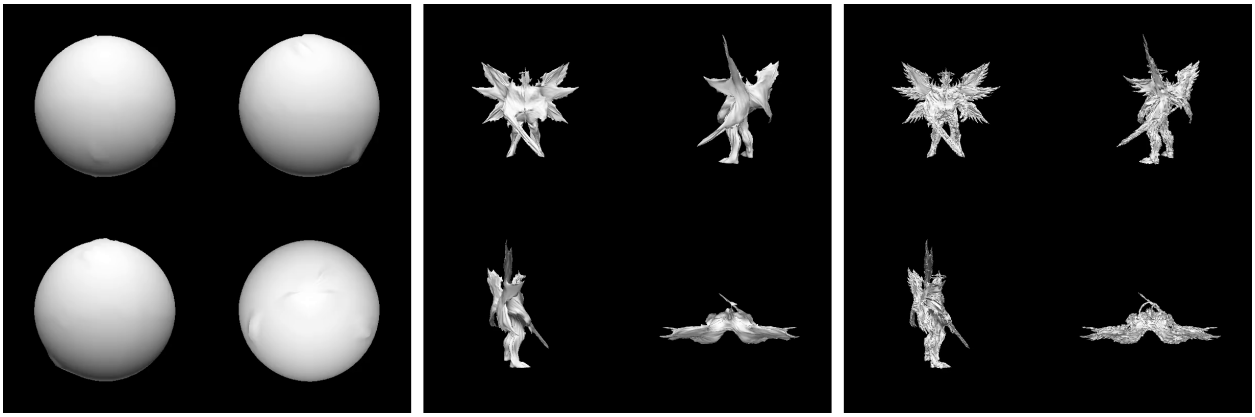


Figure 1: Evolution of optimized mesh in current C++ implementation. Texrue optimiztion not implemented yet.

The Challenge

All sections have room to improve parallelism, the challenge is being able to achieve real-time differentiable.

- Vertex gradient contention in the backward pass
- Texel gradient contention in the backward pass
- Workload distribution in kernels for high-resolution images and heavy meshes

- Load imbalance due to varying triangle sizes
- Divergent execution (inside/outside tests, closest-edge selection, margin culling)
- Irregular memory access (Laplacian neighbor gathers, vertex ordering after subdivision)
- Subdivision dependencies
- Multi-GPU communication (gradient reduction over PCIe)
- Multi-GPU vs. single-GPU batching
- Kernel launch overhead and synchronization (fused kernels, CUDA Graphs)

Resources

- **Machines:** GHC machines (NVIDIA RTX 2080) for single-GPU work; lab server with 8× NVIDIA RTX A4000 for multi-GPU experiments.
- **Code base:** own C++ CPU implementation (forward, backward, loss, Laplacian smoothing, Adam, subdivision), used as the sequential baseline and starting point for the CUDA version. Still needs texture optimization.
- **References:** SoftRas [1]; nvdiffrast [2] and PyTorch3D [3] as external baselines.
- **Test meshes:** Sketchfab
- **Tools:** Nsight Compute and Nsight Systems for profiling.

[1] S. Liu, T. Li, W. Chen, H. Li. *Soft Rasterizer: A Differentiable Renderer for Image-based 3D Reasoning.* ICCV 2019.
[2] S. Laine, J. Hellsten, T. Karras, Y. Seol, J. Lehtinen, T. Aila. *Modular Primitives for High-Performance Differentiable Rendering.* ACM TOG (SIGGRAPH Asia) 2020.
[3] N. Ravi et al. *Accelerating 3D Deep Learning with PyTorch3D.* arXiv:2007.08501, 2020.

Goals and Deliverables

Plan to Achieve

- Under 10 ms for a complete iteration over the whole batch (forward, backward, loss, regularization, optimizer step) at the 2nd subdivision level, and under 20 ms at the 3rd subdivision level (∼200K faces) with multi-GPU, for a batch of 20 views at 400×400. These targets are based on my experience profiling and optimizing 3D Gaussian Splatting with similar primitive counts, where the timings were in this range.
- At least 100× speedup on the forward and backward passes versus my single-threaded CPU implementation. The CPU version currently takes about 5 s per iteration (20 views, 400×400) with a large σ, and less with a small σ since the bounding-box margin shrinks; reaching the 10–20 ms target above requires a speedup of at least this order.
- A complete analysis, in terms of execution time, throughput, and memory bandwidth, of:

- batching on single vs. multi-GPU, including the gradient reduction over PCIe;
- atomic backpropagation contention strategies for vertex and texel gradients (memory vs. latency);
- forward and backward kernel structure strategies (shared memory, registers, warp intrinsics, ...);
- the subdivision and Laplacian smoothing kernels;
- kernel occupancy and divergence;
- a per-stage time breakdown of one iteration;
- Fused kernels
- RTX 2080 vs. RTX A4000.

Hope to Achieve

- Vertex load balancing within kernels.
- Vertex reordering after subdivision, and its effect on memory locality and performance.
- CUDA Graphs.
- Scaling with mesh resolution, image resolution, and number of views.
- Comparison against SoftRas and nvdiffrast.

Poster Session

I will present a poster and a demo that shows several render targets and how the optimization happens in real time (if I manage to obtain a real-time solution); if not, I will show a sped-up video of the optimization instead.

Platform Choice

CUDA & C++. CUDA makes it easy to program NVIDIA GPUs and has been taught in the course. I considered Vulkan with GLSL so that the project would be platform-independent, but Vulkan has a lot of overhead that would take more time than it should, considering the objectives of the project. This is essentially a GPU problem, as it is massively parallel and independent in the pixels and vertices, which is essentially what GPUs were designed for.

Schedule

Week	Tasks
9/11–15/11	Finish implementing the sequential baseline (texture optimization) and profile the CPU baseline extensively (20 views, 400×400 px). Initial CUDA forward (triangle binning) and backward kernels (atomics for gradient accumulation), plus initial loss, Laplacian smoothing, and optimizer kernels. Extensive profiling of the baseline parallel kernels (divergence, throughput, memory, occupancy, ...), comparison with the CPU version, and bottleneck detection.
16/11–22/11	Subdivision kernel implementation, single-GPU batching, backward gradient-accumulation contention strategies, and forward kernel structure strategies. Profile again. Prepare the milestone report.
23/11–29/11	Multi-GPU batching with gradient reduction over the interconnect, and RTX 2080 vs. A4000 comparison runs. Fused kernels, and scaling analysis with resolution and mesh size.
30/11–10/12	CUDA Graphs implementation, load-balancing analysis and potential solutions, vertex reordering for better locality, comparison with SoftRas (and possibly nvdiffrast). Prepare the final report, poster, and demo.